

Library Management System — Full Stack

A full-stack library management application with JWT-authenticated REST APIs, a relational data model with real entity relationships, and a React frontend — built as a portfolio project to demonstrate backend architecture beyond simple single-entity CRUD.

Built as a follow-up to a single-entity Employee Management API, this project intentionally adds multi-entity relationships, authentication, and real business logic (loan limits, overdue fines) to show a deeper understanding of backend design.

What This Project Demonstrates

- **Relational data modeling** — @ManyToOne / @OneToMany relationships across 5 entities (not just one flat table)
 - **JWT authentication** — register/login endpoints, token-based auth on protected routes
 - **Real business logic in the service layer** — can't borrow more than 3 books at once, can't issue a book with zero copies available, automatic overdue fine calculation
 - **Full-stack integration** — React frontend consuming a Spring Boot REST API over CORS, with a token-based auth flow
 - **Clean layered architecture** — Controller → Service → Repository → Entity, consistently applied across all 5 modules
-

Tech Stack

Backend

- Java 17, Spring Boot 3.x
- Spring Web (REST controllers)
- Spring Data JPA + Hibernate
- Spring Security + JWT (jjwt)
- MySQL
- Maven

Frontend

- React 18
 - React Router
 - Fetch API for backend communication
 - Custom CSS (no UI framework — hand-built "reading room" design system)
-

Architecture

React (localhost:3000) <—fetch + JWT—> Spring Boot (localhost:8080)
<—JPA/Hibernate—> MySQL (localhost:3306)

Entity relationships:

Author (1) —< (many) Book
Category (1) —< (many) Book
Member (1) —< (many) BorrowRecord >———— (1) Book

Project Structure

```
library-backend/
├── src/main/java/com/library/
│   ├── config/           # Security config, CORS, JWT filter
│   ├── controller/       # REST endpoints (6 controllers)
│   ├── dto/              # Request/response objects
│   ├── exception/        # Global exception handler
│   ├── model/            # JPA entities (Author, Category, Book, Member,
BorrowRecord, User)
│   ├── repository/       # Spring Data JPA repositories
│   ├── service/          # Business logic layer
│   └── LibraryApplication.java
├── src/main/resources/application.properties
library-management-frontend/
├── src/
│   ├── api/              # Centralized API service layer
│   ├── context/          # AuthContext (JWT + signed-in user state)
│   ├── pages/            # SignIn, SignUp, Dashboard, Books, Members,
BorrowReturn, MemberHistory
│   ├── styles/           # Component + page CSS
│   └── App.js
```

API Reference

Module	Method	Endpoint	Description
Auth	POST	/api/auth/register	Create account, returns JWT
Auth	POST	/api/auth/login	Sign in, returns JWT
Authors	GET/POST/PUT/DELETE	/api/authors, /{id}	Author CRUD
Categories	GET/POST/PUT/DELETE	/api/categories, /{id}	Category CRUD
Books	GET/POST/PUT/DELETE	/api/books, /{id}	Book CRUD
Books	GET	/api/books/search?author=&category=	Search/filter

Module	Method	Endpoint	Description
		y=	
Members	GET/POST/PUT/DELETE	/api/members/{id}	Member CRUD
Borrow	POST	/api/borrow	Issue a book (enforces business rules)
Borrow	PUT	/api/borrow/{id}/return	Return a book, calculate fine
Borrow	GET	/api/borrow/active	Currently borrowed books
Borrow	GET	/api/borrow/member/{id}	A member's borrow history

All endpoints except `/api/auth/**` require a `Authorization: Bearer <token>` header.

Business Rules Implemented

- A member **cannot borrow more than 3 books** at once
 - A book **cannot be issued** if `availableCopies` is 0
 - Returning a book past its due date **automatically calculates a fine** (₹5/day late)
 - `availableCopies` is decremented on issue and incremented on return
-

Getting Started

1. Database

```
CREATE DATABASE library_db;
```

2. Backend

```
cd library-backend
# Edit src/main/resources/application.properties – set your MySQL password
mvn spring-boot:run
```

Runs on `http://localhost:8080`. Hibernate auto-creates all tables on first run.

3. Frontend

```
cd library-management-frontend
npm install
npm start
```

Runs on `http://localhost:3000`.

4. First-time setup

1. Go to `http://localhost:3000/signup` and create an account
 2. Add a few authors and categories via Postman (include your JWT in the `Authorization` header)
 3. Add books, then members
 4. Try the borrow/return flow from the UI
-

What I'd Build Next

- Route guards on the frontend (currently all pages are open regardless of auth state)
 - Pagination on the books/members lists
 - Email notifications for overdue books
 - Docker Compose setup for one-command local startup
-

Author

Sahana Koppad [GitHub](#) · [LinkedIn](#)